SOC L1 LAB · INC-004

INC-004: Detailed Findings Report

Two persistence paths, both running as root

INCIDENT	INC-004, Cron and systemd persistence
REPORT DATE	2026-11-01
CLASSIFICATION	Lab exercise, sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	Critical

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Executive summary
2. Scope and methodology
3. Environment
4. False-positive validation
5. Attack narrative
6. Timeline
7. Evidence
8. Detection analysis
9. Root cause
10. Impact
11. Response actions
12. Recommendations
13. Lessons learned

Appendix A: Rules
Appendix B: MITRE ATT&CK
Appendix C: Evidence index
Appendix D: Glossary

Incident	INC-004, Cron and systemd persistence
Classification	Lab exercise, sanitized for publication
Activity date	2026-10-31
Report date	2026-11-01
Author	SOC L1 portfolio lab
Status	Closed, escalated
Severity	Critical
Related	`tickets/INC-004-cron-systemd.md` , `rules/inc-004-rules.md` , `docs/evidence/INC-004-*`

---

1. Executive summary

At 23:17:24 UTC, an administrator session wrote two persistence mechanisms to the endpoint:

1. `/etc/cron.d/updates` , a cron job running every minute as root.
2. `/etc/systemd/system/updates.service` + `updates.timer` , a systemd timer running a oneshot service as root, enabled at boot.

Both executed. The proof files contained:

```
uid=0(root) gid=0(root) groups=0(root)
uid=0(root) gid=0(root) groups=0(root)
```

Detection was immediate at the file layer:

Time (UTC)	Rule	Path
23:17:25.115	100129 (level 12)	`/etc/cron.d/updates`
23:17:28.122	100130 (level 10)	`/etc/systemd/system/updates.service`
23:17:28.128	100130	`/etc/systemd/system/updates.timer`
23:17:31.309	100130	`/etc/systemd/system/timers.target.wants/updates.timer`

Audit supplied the command trail: `tee` into each path, `systemctl daemon-reload` , and `systemctl enable --now updates.timer` . Sudo recorded who ran them.

A false-positive test ran first. A legitimate cron file fired 100129 twice. A legitimate `systemctl enable cron` fired 100130 once. Both rules are change monitors: they cannot tell an approved unit from a malicious one without a change ticket. That is the central operational finding of this scenario.

---

2. Scope and methodology

2.1 Scope

In scope	Out of scope
One Debian 12 VM (`soc-victim` , 10.0.2.15)	Production systems
cron 3.0, systemd 252	Real malware or C2
FIM on `/etc` , auditd execve, sudo	Employer or client data
Local files written with sudo	Internet-facing targets

2.2 Method

1. Install `cron` as a legitimate prerequisite (the cloud image did not include it).
2. Add FIM rules for `/etc/cron.d` and `/etc/systemd/system` .
3. Test the rules against legitimate cron and systemd administration.
4. Write a cron job and a systemd service/timer that run `id` as root.
5. Enable the timer, wait for both mechanisms to execute, and capture the proofs.
6. Contain, preserve evidence, and write the ticket.

2.3 Limitations

- The actor was the lab administrator with `sudo` . The scenario models an insider or a compromised admin session, not an unprivileged attacker.
- One host. No configuration management or change-ticket integration.
- The proof command is `id` . No real payload.

---

3. Environment

Component	Detail
SIEM	Wazuh 4.14.7
Endpoint	Debian 12, 1.5 GB VM
Agent	Wazuh agent 4.14.7, ID 001, `soc-victim`
Scheduler	cron 3.0, enabled and active
Service manager	systemd 252
FIM	realtime on `/etc` , `/home` , `/var/www/html`
Audit	execve (`audit-wazuh-c`), file watches (`audit-wazuh-w`)
Rules	100129 (cron), 100130 (systemd)
Actor	`analyst` (uid 1000) via `sudo`

`/etc` is monitored as a whole, so both `/etc/cron.d` and `/etc/systemd/system` are in scope. cron was installed during the exercise. The image shipped cron files from packages but no daemon.

4. False-positive validation

Run before the attack, at 23:16:52.

Legitimate action	Result
`systemctl disable cron`	no 100130 (deletion is 553, not covered)
`systemctl enable cron`	100130 on `multi-user.target.wants/cron.service`
Create `/etc/cron.d/apt-maintenance`	100129 on the added file
Modify the same cron file	100129 on the modified file

Three legitimate alerts, no unexpected ones. The rules behave exactly as designed: they report change, not intent.

Operational consequence:

- 100129 must be matched to a change ticket before escalation. A legitimate maintenance job looks identical to a malicious one.
- 100130 fires on every `systemctl enable` because it creates a symlink under `/etc/systemd/system/*.wants/`. This is a high-volume false-positive source in a real estate.
- Deletions are not covered. Removing a unit or cron file produces 553, which these rules ignore.

5. Attack narrative

5.1 Cron

```
echo "* * * * * root /usr/bin/id > /tmp/cron-proof 2>&1" \
  | sudo tee /etc/cron.d/updates
```

Hash:

```
991523eaf5c220a6217557fafc3be04d13f5e496c85f1944641efd71fca05651
```

The job runs every minute as root. The proof file was written at 23:18:01.

5.2 systemd

```
# /etc/systemd/system/updates.service
[Unit]
Description=Updates

[Service]
Type=oneshot
ExecStart=/bin/sh -c "id > /tmp/systemd-proof 2>&1"
```

```
# /etc/systemd/system/updates.timer
[Unit]
Description=Updates timer

[Timer]
OnBootSec=30s
OnUnitActiveSec=60s
Unit=updates.service

[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now updates.timer
```

Hashes:

```
1674fae2cbf55c8e5cbb3bb8889409b951811e5a1c26c3e8f35c9efc6760c574  updates.service
b58a06d878ddb229f8bfed5eaebf1d32341ee345d0897b765868f7d16c6f0129  updates.timer
```

The timer executed the service as root. `systemctl enable` also created `/etc/systemd/system/timers.target.wants/updates.timer`, which is itself a FIM event.

---

6. Timeline

Time (UTC)	Source	Event
23:16:52	test	False-positive run starts
23:16:58.176	100130	Legit `cron.service` enable symlink
23:17:01.096	100129	Legit `/etc/cron.d/apt-maintenance` added
23:17:05.104	100129	Legit cron file modified
23:17:24	attack	Persistence run starts
23:17:25.115	100129	`/etc/cron.d/updates` added
23:17:25.887	audit 80792	`tee /etc/cron.d/updates`
23:17:28.122	100130	`updates.service` added
23:17:28.128	100130	`updates.timer` added
23:17:28.552	audit 80792	`tee updates.service`
23:17:28.568	audit 80792	`tee updates.timer`
23:17:31.309	100130	`timers.target.wants/updates.timer` symlink
23:17:31.812	audit 80792	`systemctl daemon-reload`
23:17:31.950	audit 80792	`systemctl enable --now updates.timer`
23:18:01	proof	cron and systemd both run `id` as root
23:19	response	Timer disabled, units and cron file removed

7. Evidence

7.1 File hashes

File	SHA-256
`/etc/cron.d/updates`	`991523eaf5c220a6217557fafc3be04d13f5e496c85f1944641efd71fca05651`
`/etc/systemd/system/updates.service`	`1674fae2cbf55c8e5cbb3bb8889409b951811e5a1c26c3e8f35c9efc6760c574`
`/etc/systemd/system/updates.timer`	`b58a06d878ddb229f8bfed5eaebf1d32341ee345d0897b765868f7d16c6f0129`

7.2 Execution proof

```
cron:    uid=0(root) gid=0(root) groups=0(root)
systemd: uid=0(root) gid=0(root) groups=0(root)
```

7.3 Command trail

```
23:17:25Z  sudo tee /etc/cron.d/updates
23:17:25Z  tee /etc/cron.d/updates
```

```
23:17:28Z  sudo tee /etc/systemd/system/updates.service
23:17:28Z  sudo tee /etc/systemd/system/updates.timer
23:17:31Z  sudo systemctl daemon-reload
23:17:31Z  sudo systemctl enable --now updates.timer
23:18:01Z  sh -c /usr/bin/id > /tmp/cron-proof 2>&1
23:18:01Z  /usr/bin/id
```

Alert volume in the window: 279 alerts, dominated by audit command events and motd noise. The four persistence alerts are the signal.

---

8. Detection analysis

8.1 What fired

Rule	Trigger	Latency	Value
100129	`/etc/cron.d/` file added/modified	<1 s	Persistence candidate
100130	`/etc/systemd/system/` change	<1 s	Unit or enable-link candidate
80792	`tee`, `systemctl`	~1 s	Who wrote it and how
5402	sudo command	~1 s	The actor

8.2 What the rules cannot tell you

- Whether the change was approved.
- Whether the unit is a symlink (legit enable) or a new file (candidate).
- Whether the cron file was installed by a package or written by a person.
- Whether the service actually ran, or what it ran as.

The audit trail answers the first and last of those. Change control answers the rest.

8.3 Gaps

Gap	Consequence
User crontabs in `/var/spool/cron/crontabs` are not watched	A user-level cron job is invisible
Deletions (553) are not covered	Removing persistence does not alert
100130 fires on enable symlinks	High false-positive volume
No sequence rule for unit write to enable to execute	The analyst joins them manually
No allow-list of approved units	Every legitimate unit looks the same

---

9. Root cause

9.1 Direct cause

A root-capable administrator wrote a cron file and a systemd service/timer without a change request. Both mechanisms execute as root and survive a reboot.

9.2 Contributing factors

Factor	Effect
No change control for scheduler/service changes	Nothing blocked or challenged the write
`sudo` available to the actor	Root file writes were possible
No allow-list of approved units	Legitimate and malicious units are indistinguishable
Deletion not monitored	Cleanup is invisible
Enable symlinks noisy	Analysts may start ignoring 100130

9.3 Five whys

1. Why did the persistence run? A cron file and a systemd timer were written and enabled.
2. Why were they allowed? The actor had root via sudo and no approval gate.
3. Why did they look normal? Unit files and cron jobs are ordinary administration.
4. Why was the signal noisy? 100130 also catches every legitimate `systemctl enable`.
5. Why was the response slow? The rules report changes, not authorization.

---

10. Impact

Dimension	Assessment
Confidentiality	Root execution. The host is fully exposed
Integrity	Persistence written. Boot-time execution
Availability	None observed
Blast radius	One host. Root persistence can pivot
Persistence	High: cron and systemd both survive reboot

Critical because the payload runs as root and persists. The only mitigating fact is that the actor already had sudo. No privilege escalation was required.

---

11. Response actions

1. Disabled `updates.timer` and removed the unit files.
2. Removed `/etc/cron.d/updates`.
3. Ran `systemctl daemon-reload`.

4. Recorded all three file hashes and the proof files before deletion.
5. Preserved the FIM alerts, audit commands, and sudo records.
6. Escalated for a change-control and sudo-access review.

---

12. Recommendations

Immediate

#	Action
1	Remove the units and cron file. Disable the timer
2	Hash the files before deletion
3	Check for other units, timers, and cron entries
4	Review sudo access for the actor
5	Confirm whether any change was approved

Short term

#	Action
6	Deploy 100129/100130. Route both to change-aware triage
7	Add `/var/spool/cron/crontabs` to FIM/audit coverage
8	Add deletion (553) rules for persistence paths
9	Allow-list approved unit names and cron jobs
10	Alert on `systemctl enable` for unexpected services

Long term

#	Action
11	Configuration management for units and cron. No manual writes
12	Least-privilege sudo, time-bound elevation
13	Detection-as-code tests for unit/cron writes
14	Tabletop a root-persistence incident
15	Review boot-time execution paths

---

13. Lessons learned

- **Change, not intent**
  FIM can prove a file appeared. Only change control can say whether it should have.
- **Enable symlinks are noise**
  100130 needs routing/allow-listing, not suppression.

- **Deletion is part of the story**
  Removal of persistence should also alert.
- **Root persistence is Critical**
  A cron job or timer that runs as root is a durable compromise, even if the initial action was "just sudo".
- **The proof matters**
  `uid=0` in the proof file is the difference between a suspicious file and an executed root payload.

Appendix A: Rules

```
<rule id="100129" level="12">
  <if_sid>550,554</if_sid>
  <match>/etc/cron.d/</match>
  <description>FIM: cron file added or modified (possible persistence)</description>
  <mitre><id>T1053.003</id></mitre>
  <group>syscheck,persistence,</group>
</rule>

<rule id="100130" level="10">
  <if_sid>550,554</if_sid>
  <match>/etc/systemd/system/</match>
  <description>FIM: systemd unit or enable-link changed (possible persistence)</description>
  <mitre><id>T1543.002</id></mitre>
  <group>syscheck,persistence,</group>
</rule>
```

Appendix B: MITRE ATT&CK

Technique	ID	Evidence
Scheduled task/job: cron	T1053.003	`/etc/cron.d/updates`
Create or modify system process: systemd service	T1543.002	`updates.service` / `updates.timer`
Abuse elevation control: sudo	T1548.003	`sudo tee`, `sudo systemctl`
Valid accounts	T1078	Actor `analyst`

Appendix C: Evidence index

File	Contents
`docs/evidence/INC-004-alerts.json`	279 alerts in the window
`docs/evidence/INC-004-commands.txt`	Decoded tee/systemctl/id commands
`docs/evidence/INC-004-hashes.txt`	SHA-256 for the three persistence files
`docs/evidence/INC-004-proof.txt`	Root execution proof
`docs/evidence/INC-004-files.txt`	Unit and cron file contents

Appendix D: Glossary

Term	Meaning
`cron.d`	System-wide cron drop-in directory. Files run as specified
systemd timer	Scheduled unit that triggers a service
`enable` symlink	`/etc/systemd/system/*.wants/` link created by `systemctl enable`
FIM	File integrity monitoring
FIM 553	File deleted event